营销活动

优惠券 / 秒杀 / 抢红包——三个 Lua 套路对照与防黑产

gomall · 业务侧 deck

今日大纲

把三个场景拼到一张桌子上

优惠券：库存 + 限领二维原子

秒杀：滑动窗口 + 单独商品表

抢红包：LPOP 一次取一份

防黑产：分层与现状

把三个场景拼到一张桌子上

优惠券 / 秒杀 / 抢红包凭什么放一起讲

- 表面三类活动，骨架其实是同一套：
  - 入口流量打到一个共享资源（库存 / 商品份额 / 红包份数）。
  - 必须原子扣减，否则超发 = 直接资金损失。
  - 防刷需求强：羊毛党盯的就是这种” 低价 / 免费” 场景。
  - 抢到只是第一步，还要落库 + 入账，链路天然要 Saga 兜底。
- gomall 三个接口的代码其实长得像三胞胎：都是**中间件链 → Lua 一锤 → 事务落库 → outbox 投递**。
- 本 deck 把三段 Lua 摆在一起读，挑差异、说原因。

对应 *service/coupon.go, service/redpacket.go, service/skill_goods.go*

统一链路 + 三个路由的中间件链

活动	路由	中间件链
优惠券	`POST /coupon/claim`	(仅全局 TokenBucket)
秒杀	`POST /skill_product/skill`	SlidingWindow{seckill, 3/s, ByUser}
抢红包	`POST /redpacket/claim`	SlidingWindow{redpacket, 3/s, ByUser} + Idempotency

routes/router.go:22 全局桶；*:72 coupon*；*:79–84* 红包；*:136–143* 秒杀

优惠券：库存 + 限领二维原子

coupon Lua 脚本（业务画像 + 10 行实现）

`coupon:batch:{id}:stock` 整型库存；`coupon:user:{uid}:batch:{id}` 单人计数 TTL 24h。
Lua 必须一锤完成：库存 > 0？ 单人 < `PerUser`？ 两道闸过了才 DECR + INCR。

Listing 1: *repository/cache/coupon.go:30–44 claimScript*

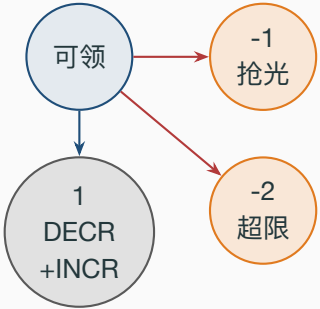
```lua
local stock = tonumber(redis.call('GET', KEYS[1]))
if stock == nil or stock <= 0 then
   return -1 -- 已抢光
end
local owned = tonumber(redis.call('GET', KEYS[2]))
if owned == nil then owned = 0 end
if owned >= tonumber(ARGV[1]) then
   return -2 -- 超出单人上限
end
redis.call('DECR', KEYS[1])
redis.call('INCR', KEYS[2])
redis.call('EXPIRE', KEYS[2], tonumber(ARGV[2]))
return 1
```

Lua 脚本逐行解读

- **L1–L4**：读库存，**nil 当 0 处理**。返回 `-1` 给业务层翻译成” 已抢光”。
- **L5–L9**：读单人计数。第一次领该券时 key 不存在，Lua 里 `nil` 强转 0；命中上限直接返回 `-2`。
- **L10–L11**：DECR / INCR 在 Lua 里是同一个事务原子单位，不会出现” 扣了库存但没记上单人”。
- **L12**：每次领到都续命 24h ——防止 key 在活动跨天时被 evict 导致重新可领。
- 三个 return 码业务层映射成两种业务错误（见 `ClaimCouponAtomic`）。

repository/cache/coupon.go:57–75 错误码翻译

二维原子图 + REPORT 数字



	Redis Lua	DB 锁
500 并发抢 100 张	零超发	零超发
吞吐 RPS	51,362	50,142
avg 延迟	1.24ms	1.29ms
max 延迟	**136ms**	**453ms**

两

道闸都要原子：先 DECR 再判单人，已超限的请求会留” 幽灵库存”。
吞吐拉平，但 Lua vs FOR UPDATE **尾延迟差 3.3 倍**。*repository/cache/coupon.go:30–44*；*stressTest/REPORT.md* 链路压测表第 *5–6* 行

coupon 的盲区：Lua 防超发但不防黑产

- Lua 看的是 `userId+batchId`：一个账号最多领 `PerUser` 张。
- 但羊毛党的玩法是**一万个账号每号领一张**，Lua 看不出来。
- 现状：router 上只有全局 TokenBucket 100 RPS / IP，单 IP 可以慢慢刷。
- 改进建议：追加 `SlidingWindow{Scope:"coupon", Limit:5, Window:1m, ByUser:true}`。
  - 真用户 1min 领 5 张够用。
  - 万号脚本：要么换号慢，要么被网关 TokenBucket 截掉。

routes/router.go:72 coupon/claim 路由（无单用户限频）

秒杀：滑动窗口 + 单独商品表

秒杀和” 普通下单” 的根本差异

- 库存独立：`model.SkillProduct` 与 `Product` 分表，不污染主商品池。
- 限流前置：`/skill_product/skill` 强制挂 SlidingWindow，单用户 1s 最多 3 次。
- 没有熔断：秒杀打开熔断 = ” 所有人都买不到”，业务等价于” 活动作废”，比限流损失更大。
- 没有 outbox：当前实现**只读缓存**，扣库存的真实路径留在 deck 11（库存业务）。

service/skill_goods.go:122–134 SkillProduct；routes/router.go:136–143

SlidingWindow Lua 怎么算窗口 + 阈值取 3 的依据

member = 请求 id，score = 毫秒时间戳。Lua 先 `ZREMRANGEBYSCORE` 清窗外旧记录，再 `ZCARD` 判定。

阈值 = 3：真人 300–500ms 一次，3 留 50% 余量；取 10 形同没限，取 1 网络抖一下客诉爆。**比真人快、比脚本慢**，没有第二候选。*repository/cache/ratelimit.go:18–34*；*routes/router.go:136–143*

REPORT 数字：3/s/用户误差 2.2%

场景：30 VU × 15s 打 `/skill_product/skill`，单 token，配置 `Limit=3 Window=1s ByUser=true`。

指标	实测	业务期望
通过请求数	46	15s × 3 = 45
被限流数	781,624	黑产应当被全部拦下
误差	2.2%	< 5% 视为合格

”通过 46 次” 约等于” 真用户最快也只能秒到 45 次/15s”，业务侧合理。

被截 78 万次都是同一 token，正是黑产的标准画像。*stressTest/REPORT.md §5* 限流中间件

抢红包：LPOP 一次取一份

为什么红包不能用 DECR：发包时预拆 + 二倍均值法

业务约束：总额严格守恒（100 元发出去必须正好 100 元），每份金额不同 → DECR 拿不回金额，必须 LPOP 出队拿” 那一份”。**发包时一次性拆好**，写入 Redis LIST。

Listing 2: *repository/cache/redpacket.go:107–141 SplitRedPacket*

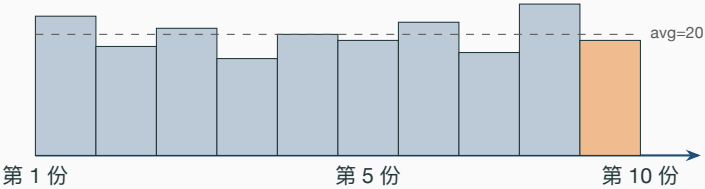
```go
out := make([]int64, count)
remain := total
for i := 0; i < count-1; i++ {
   left := int64(count - i)
   avg := remain / left * 2 // 二倍均值
   if avg < 2 { avg = 2 }
   maxThisRound := remain - int64(count-1-i) // 留至少 1 分给后面每份
   hi := avg - 1
   if hi > maxThisRound { hi = maxThisRound }
   if hi < 1 { hi = 1 }
   amt := splitRand.r.Int63n(hi) + 1 // [1, hi]
   out[i] = amt
   remain -= amt
}
out[count-1] = remain // 最后一份兜底
```

二倍均值逐行解读

- **avg = remain/left * 2**：剩余金额除以剩余份数 × 2，作为”本次最大可分到金额”的基础。
- **maxThisRound**：留给后面每份至少 1 分（避免最后几份被迫为 0）。
- **Int63n(hi)+1**：在 `[1, hi]` 闭区间随机，保证每份至少 1 分。
- **out[count-1] = remain**：最后一份直接吃剩，确保总和守恒。
- **Shuffle**：不打散的话，越靠前抢到的期望金额越大——用户会推断”第一个抢的最大”。
- **splitRand 进程内独占 rand**：`math/rand` 全局源加锁，热点路径并发不友好。

repository/cache/redpacket.go:92–142

二倍均值法分布示意（200 元 / 10 份）



每份在 `[1, 2*avg-1]` 之间

随机，期望约等于 avg，方差由 hi 上界控制。
末位（橙色）是 `remain` 兜底，shuffle 后位置随机。*repository/cache/redpacket.go:113–141*

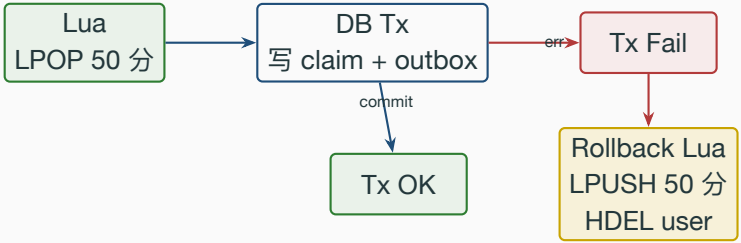
抢红包 Lua 脚本（8 行）

Listing 3: *repository/cache/redpacket.go:52–63 claimRedPacketScript*

```lua
if redis.call('HEXISTS', KEYS[2], ARGV[1]) == 1 then
   return -2 -- 已领过
end
local amt = redis.call('LPOP', KEYS[1])
if not amt then
   return -1 -- 已抢完
end
redis.call('HSET', KEYS[2], ARGV[1], amt)
redis.call('EXPIRE', KEYS[2], tonumber(ARGV[2]))
return tonumber(amt) -- 抢到金额
```

HEXISTS 前置防同用户多抢；**LPOP 而非 BLPOP** 抢完即 `-1`（业务码 RP-EMPTY）不阻塞连接；**return amt** 省一次 `HGET` 回查。与 coupon 对比：coupon 是对称扣减（计数器），红包是资源出队（队列）。

抢红包 Saga：Lua 成功但 DB 失败怎么办



把已 LPOP 的金额 `LPUSH` 回队头、同时撤销 `claimed` 标记，用户下次再抢可以正常领。

对应 `rollbackRedPacketClaimScript`：两个动作放在一个 Lua 原子单元。

repository/cache/redpacket.go:86–90 rollback；service/redpacket.go:155–162 调用

三个 Lua 在做什么：维度对照 + 选型逻辑

维度	优惠券	秒杀（限流）	抢红包
核心命令	DECR + INCR	ZADD + ZCARD	LPOP + HSET
数据结构	STRING × 2	ZSet	LIST + HASH
原子单元	库存 + 单人计数	时间窗口集合	出队 + 去重标记
预热成本	一次 SET 库存	无（按需建窗口）	**发包时全量预拆**
TTL 策略	用户标记 24h	窗口大小 1s	claimed hash > 红包过期
回滚	INCR + DECR	无（不可逆）	LPUSH + HDEL

反过来用就不行：红包用 DECR → 并发算金额不守恒；优惠券用 LPOP → 浪费；限流用 DECR → 固定窗口边界 2× 突刺。

业务语义决定结构选型：金额异质 → 队列；同质 → 计数器；时间窗口 → ZSet。

repository/cache/coupon.go, ratelimit.go, redpacket.go 三处实现

三段 Lua 的共同设计纪律

1. **失败也要原子**：判断条件不通过时**什么都不写**，直接 return 错误码（-1 / -2）。
2. **回滚也是 Lua**：DB 失败时的补偿动作要走**同一种原子机制**，不能 Go 侧两次 Redis 调用。
3. **TTL 永远要设**：尤其是抢/领标记类 key，否则用户停留态 key 会无限堆积。
4. **返回值留语义**：用 `>0` / `0` / 负数区分成功金额、抢完、已抢过、超限，业务层 switch 即可。
5. **进程内 rand**：发包拆分时**不要用 math/rand 全局**，热点路径并发争锁。

repository/cache/redpacket.go:92–96 splitRand 局部源；*coupon.go:30–44* 错误码语义

防黑产：分层与现状

为什么营销是黑产首选 + 四层防线

黑产为何盯营销：真金白银（券能转卖、红包能提现）+ 规则透明（必须对外通告）+ 多账号便宜（手机号能买）。本质是**把作弊单价拉到 > 收益**。

层级	工具	gomall 现状	备注
L1 网络	Anti-DDoS / CDN 黑名单	未覆盖	依赖云厂商
L2 网关	IP TokenBucket	已接	router.go:22
L3 业务	用户 SlidingWindow	部分	秒杀 + 红包，coupon 缺
L4 风控	设备指纹 / 行为模型	路线图	接外部风控

每多一层，黑产成本翻倍：

刷券既要换 IP（L2）、又要换号（L3）、又要换设备指纹（L4），单券薅羊毛就不划算了。gomall 站住 L2 + L3。*routes/router.go:22* 全局 *IP* 桶；*:79–84, :136–143* 用户滑动窗口

IP TokenBucket vs 用户 SlidingWindow 选哪个

维度	TokenBucket (L2)	SlidingWindow (L3)
key 维度	ClientIP	userID
精度	桶内突发可过冲	严格窗口
绕过成本	换 IP / 走代理	换账号
位置	全局，所有请求	营销 / 高敏接口

TokenBucket 挡的是” 机器自动化”，SlidingWindow 挡的是” 账号刷子”。

两层一起上，黑产**必须同时凑齐 IP 池 + 账号池**才能突破，门槛指数级抬高。

middleware/ratelimit.go TokenBucket vs SlidingWindow

客服话术：业务码到提示文案

状态码	含义	客服话术
70001	限流命中（SlidingWindow）	” 操作过于频繁，请稍后再试”
70002	熔断打开（CB-OPEN）	” 支付服务暂忙，1 分钟后”
60002	幂等命中	” 您的操作已受理”
50001	库存不足	” 已售罄” / RP-EMPTY
-2 (coupon)	单人超限	” 每人限领 N 张”

业务侧约

定：70001 不是失败，是” 稍后重试就行”；RP-EMPTY 是终态，告诉用户红包已被抢完。客服话术必须**区分**两种” 稍后再试”，否则用户白等。*pkg/e/code.go:53–58* 业务码定义

三个活动的 SLO 与各角色视图

活动	可用率	p99	备注
优惠券领取	99.9%	< 200ms	大促日双倍预热库存
秒杀下单	99%	< 200ms	反黑产强需求，可接受偶发误伤
抢红包	99.9%	< 300ms	资金敏感，Saga 必须兜底

角色	关心指标	数据来源
运营	用券率 / 红包领完率	coupon batch + finished 数
风控	70001 / 60002 占比	业务码统计
客服	投诉单 □ 业务码反查	status 字段
SRE	Lua RTT p99 / Redis CPU	监控大盘
财务	红包总额守恒	DB sum vs outbox

秒杀可用率最低——” 全场抢光” 是营销目标，5xx 短抖动不算事故。红包反过来：抢到没入账 = 投诉。70001 比例上升，运营看的是” 参与热度”，风控看的是” 被刷”。对照 *deck 10 P0–P3* 等级表；*pkg/e/code.go*

面试 Q&A（上）

Q1：为什么 coupon 用 DECR，红包用 LPOP?

A：券同质，整型计数器即可；红包每份金额不同，必须出队拿” 那一份”。

Q2：coupon Lua 里” 先查库存再查单人” 能不能换顺序?

A：可以换，但语义有别：先查单人会先消耗 user key 的读，库存爆满时反而更慢。当前顺序对” 已抢光” 这种最常见的失败短路更划算。

Q3：3/s/用户这个数从哪来?

A：真人点击 300–500ms 一次，3 留 50% 余量。压测 46/45 误差 2.2%。

面试 Q&A（下）

Q4：红包 Lua 已经 LPOP 了，DB 落库失败怎么办？

A：Saga 回滚——跑 `rollbackRedPacketClaimScript`，`LPUSH` 金额回队头 + `HDEL` 撤销 claimed 标记，用户下次能再抢。

Q5：羊毛党用一万个账号刷 coupon，gomall 现在能挡住吗？

A：挡不住。需要追加 `SlidingWindow{coupon, 5/min, ByUser}`。当前只能靠全局 TokenBucket 100 RPS/IP 拖速度。

Q6：秒杀为什么不挂 Idempotency？

A：秒杀重复点击的语义就是” 再试一次”，幂等命中反而误导用户。限流截掉就够了。

代码位置一览

coupon Lua + 错误码 *repository/cache/coupon.go:30–75*

coupon 业务层 *service/coupon.go:65–99 Claim*

红包拆分算法 *repository/cache/redpacket.go:102–142 SplitRedPacket*

红包 claim Lua *repository/cache/redpacket.go:52–63*

红包 Saga 回滚 *repository/cache/redpacket.go:86–90 + service/redpacket.go:155–162*

秒杀业务 *service/skill_goods.go:122–134*

秒杀路由 + 滑动窗口 *routes/router.go:136–143*

红包路由 + 滑动窗口 + 幂等 *routes/router.go:79–84*

SlidingWindow Lua *repository/cache/ratelimit.go:18–34*

业务码 *pkg/e/code.go:53–58*

配套技术 deck：05-ratelimit / 10-traffic-governance / 16-redpacket